Лабораторная работа №3

Пратько Максим 453503 вариант 23

Индивидуальные задания по вариантам

Задание 1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд. Задать точность вычислений eps.

Предусмотреть максимальное количество итераций, равное 500.

Вывести количество членов ряда, необходимых для достижения указанной точности вычислений. Результат получить в виде:

x	n	$F(x)$	$Math\ F(x)$	eps

Здесь x – значение аргумента, F(x) – значение функции, n – количество просуммированных членов ряда, Math F(x) – значение функции, вычисленное с помощью модуля math.

$$\ln(1+x) = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} + \ldots, |x| < 1$$

```
# Description: Computes ln(1+x) using the Taylor series expansion
# and compares the result with math.log.

import math
from utils import get_float, timing_decorator

MAX_ITERATIONS = 500

@timing_decorator
def compute_series(x: float, eps: float) -> tuple[float, int]:
"""
Compute ln(1+x) via the alternating Taylor
Iteration stops when |term| < eps or MAX_ITERATIONS is reached.

Args:
x: The argument; must satisfy |x| < 1.
eps: Desired precision (stopping criterion).
```

```
Returns:
A tuple (approximation, iterations_used).

Raises:
ValueError: If |x| >= 1 (series does not converge).
"""
if abs(x) >= 1:
raise ValueError("The series converges only when |x| < 1.")

total = 0.0
term = x
n = 1

while n <= MAX_ITERATIONS:
total += term
if abs(term) < eps:
break
n += 1
term = ((-1) ** (n - 1)) * (x ** n) / n

return total, n

def print_series_result(x: float, eps: float,
approx: float, exact: float, iters: int) -> None:
"""
Display a formatted comparison table for the series result.

Args:
x: Input value.
eps: Precision used.
approx: Series approximation of ln(1+x).
exact: Exact value from math.log.
iters: Number of iterations performed.
"""
header = f"{'x':>10} {'n':>6} {'F(x)':>20} {'math.log(1+x)':>20} {'eps':>12}"
separator = "-" * len(header)
row = (f"{x:>10.6f} {iters:>6} "
f"{approx:>20.10f} {exact:>20.10f} {eps:>12.2e}")

print("\n Results:")
print(" " + header)
print(" " + separator)
```

```
print(" " + row)

if iters >= MAX_ITERATIONS:
print(f"\n Warning: reached the maximum of {MAX_ITERATIONS} iterations.")
print(" The required precision may not have been achieved.")


def task1() -> None:
"""
Business function for Task 1.

Prompts the user for x and eps, computes ln(1+x) via power series,
and prints a comparison table with the exact math.log value.
"""
print(" TASK 1 — Compute ln(1+x) via Taylor series")

x = get_float(" Enter x (|x| < 1): ", min_val=-0.9999999, max_val=0.9999999)
eps = get_float(" Enter precision eps (e.g. 1e-6): ", min_val=1e-15, max_val=1.0)

try:
approx, iters = compute_series(x, eps)
except ValueError as exc:
print(f" Computation error: {exc}")
return

exact = math.log(1 + x)
print_series_result(x, eps, approx, exact, iters)
```

```
 MAIN MENU

 1. Task 1
 2. Task 2
 3. Task 3
 4. Task 4
 5. Task 5
 0. Exit

 Select task (0 to exit): 1
   TASK 1 — Compute ln(1+x) via Taylor series
   Enter x (|x| < 1): 0.2
   Enter precision eps (e.g. 1e-6): 1e-6
 [Timer] 'compute_series' finished in 0.000022 seconds.

   Results:
            x       n                F(x)        math.log(1+x)          eps
   ------------------------------------------------------------------------
     0.200000       8          0.1823215086         0.1823215568      1.00e-06
```

Задание 2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел.

Организовать цикл, который принимает целые числа и находит количество чисел, больших числа 23. Окончание – ввод 15

```
# Description: Reads integers from the user until the sentinel value 15
# is entered, then counts how many numbers exceed 23.

from utils import get_int

SENTINEL = 15
THRESHOLD = 23

def collect_numbers() -> list[int]:
"""
Collect integers from the user until the sentinel value (15) is entered.

Returns:
A list of all integers entered before the sentinel.
"""
numbers: list[int] = []
print(f" Enter integers (enter {SENTINEL} to finish):")

while True:
value = get_int(" Number: ")
numbers.append(value)
if value == SENTINEL:
break
return numbers

def count_above_threshold(numbers: list[int], threshold: int) -> int:
"""
Count how many values in *numbers* are strictly greater than *threshold*.

Args:
numbers: List of integers to examine.
threshold: The comparison boundary.

Returns:
The count of elements where element > threshold.
```

```
"""
return sum(1 for n in numbers if n > threshold)

def task2() -> None:
"""
Business function for Task 2.

Collects integers from the user (stopping at sentinel 15),
then reports how many of them are greater than 23.
"""
print(" TASK 2 — Count numbers greater than 23")

numbers = collect_numbers()

if not numbers:
print(" No numbers were entered (only the sentinel was received).")
return

result = count_above_threshold(numbers, THRESHOLD)
print(f"\n Numbers entered: {numbers}")

print(f" Count of numbers greater than {THRESHOLD}: {result}")
```

```
Select task (0 to exit): 2
  TASK 2 — Count numbers greater than 23
  Enter integers (enter 15 to finish):
  Number: 25
  Number: 24
  Number: 4
  Number: 15

  Numbers entered: [25, 24, 4, 15]
  Count of numbers greater than 23: 2
```

Задание 3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры.

Определить, является ли введенная с клавиатуры строка шестнадцатеричным числом

```
# Description: Determines whether a user-supplied string represents
# a valid hexadecimal number.

_HEX_CHARS = frozenset('0123456789ABCDEFabcdef')

def is_hex_string(s: str) -> bool:
```

```
"""
Check whether *s* is a non-empty string containing only hexadecimal digits.

Valid hex digits are: 0-9, A-F, a-f (case-insensitive).

Args:
s: The string to validate.

Returns:
True if *s* is a valid hexadecimal string, False otherwise.

Raises:
TypeError: If *s* is not a string.
"""
if not isinstance(s, str):
raise TypeError(f"Expected str, got {type(s).__name__}.")
if not s:
return False
return all(ch in _HEX_CHARS for ch in s)

def get_nonempty_string(prompt: str) -> str:
"""
Prompt the user for a non-empty string, repeating until one is provided.

Args:
prompt: Text displayed to the user.

Returns:
A non-empty string entered by the user.
"""
while True:
value = input(prompt).strip()
if value:
return value
print(" Error: input cannot be empty. Please try again.")

def task3() -> None:
"""
Business function for Task 3.

Reads a string from the user and reports whether it is a valid
hexadecimal number.
"""
```

```
print(" TASK 3 — Check whether a string is hexadecimal")

text = get_nonempty_string(" Enter a string: ")

try:
if is_hex_string(text):
print(f" Result: '{text}' IS a valid hexadecimal number.")
else:
print(f" Result: '{text}' is NOT a valid hexadecimal number.")
except TypeError as exc:
print(f" Type error: {exc}")
```

```
Select task (0 to exit): 3
  TASK 3 — Check whether a string is hexadecimal
  Enter a string: abcd56
  Result: 'abcd56' IS a valid hexadecimal number.
```

Задание 4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы:

«So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.»

Если не оговорено иное, то регистр букв при решении задачи не имеет значения.

а) определить количество слов, состоящих из прописных букв ; б) найти самое длинное слово, которое начинается на букву 'l'; в) вывести повторяющиеся слова

```
# Description: Analyses a fixed text passage to find:

# 1) Words consisting entirely of capital letters.
# 2) The longest word starting with the letter 'l'.
# 3) All words that appear more than once (with their frequencies).

_SOURCE_TEXT = (
"So she was considering in her own mind, as well as she could, "
"for the hot day made her feel very sleepy and stupid, whether the pleasure of "
"making a daisy-chain would be worth the trouble of getting up and picking the "
"daisies, when suddenly a White Rabbit with pink eyes ran close by her."
)


def clean_word(word: str) -> str:
"""
```

```
Strip leading/trailing punctuation from *word* and convert to lower case.

Args:
word: A raw token from the text.

Returns:
The cleaned, lower-cased word.
"""
return word.strip(',.!?;:()"-').lower()

def tokenize(text: str) -> list[str]:
"""
Split *text* into a list of raw word tokens.

Args:
text: The full source string.

Returns:
List of non-empty whitespace-separated tokens.
"""
return [w for w in text.split() if w]

def find_all_caps_words(raw_tokens: list[str]) -> list[str]:
"""
Return tokens that consist solely of uppercase alphabetic characters.

Args:
raw_tokens: Whitespace-split tokens (punctuation still attached).

Returns:
List of fully-uppercase words.
"""
return [w for w in raw_tokens if w.isupper() and w.isalpha()]

def find_longest_l_word(cleaned_tokens: list[str]) -> str | None:
"""
Return the longest word (cleaned, lowercase) that starts with 'l'.

Args:
cleaned_tokens: Tokens already stripped of punctuation and lower-cased.

Returns:
```

```
The longest 'l'-word, or None if no such word exists.
"""
l_words = [w for w in cleaned_tokens if w.startswith('l')]
return max(l_words, key=len) if l_words else None

def find_repeated_words(cleaned_tokens: list[str]) -> dict[str, int]:
"""
Build a frequency map of words that appear more than once.

Args:
cleaned_tokens: Tokens already stripped of punctuation and lower-cased.

Returns:
Dictionary mapping each repeated word to its occurrence count,
sorted by frequency in descending order.
"""
freq: dict[str, int] = {}
for word in cleaned_tokens:
if word:
freq[word] = freq.get(word, 0) + 1

repeated = {w: c for w, c in freq.items() if c > 1}
return dict(sorted(repeated.items(), key=lambda kv: kv[1], reverse=True))

def task4() -> None:
"""
Business function for Task 4.

Analyses the built-in text passage and prints:
• count of all-caps words,
• longest word starting with 'l',
• all repeated words with their frequencies.
"""
print(" TASK 4 — Text analysis")
print("\n Source text:")
print(f" {_SOURCE_TEXT}\n")

raw_tokens = tokenize(_SOURCE_TEXT)
cleaned_tokens = [clean_word(w) for w in raw_tokens]

# --- 1) All-caps words ---
caps_words = find_all_caps_words(raw_tokens)
print(f" 1) Words in ALL CAPS: {len(caps_words)} → {caps_words}")
```

```
# --- 2) Longest word starting with 'l' ---
longest_l = find_longest_l_word(cleaned_tokens)
if longest_l:
print(f" 2) Longest word starting with 'l': '{longest_l}'")
else:
print(" 2) No words starting with 'l' were found.")

# --- 3) Repeated words ---
repeated = find_repeated_words(cleaned_tokens)
print(" 3) Repeated words:")
if repeated:
for word, count in repeated.items():
print(f" '{word}' — {count} times")
else:
print(" No repeated words found.")
```

```
TASK 4 — Text analysis

Source text:
So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-c
hain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.

1) Words in ALL CAPS: 0  → []
2) No words starting with 'l' were found.
3) Repeated words:
     'the' — 4 times
     'her' — 3 times
     'she' — 2 times
     'as' — 2 times
     'and' — 2 times
     'of' — 2 times
     'a' — 2 times
```

Задание 5. В соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции:

1) ввод элементов списка пользователем;

2) проверка корректности вводимых данных;

3) реализация основного задания с выводом результатов;

4) вывод списка на экран.

В списке, состоящем из целых элементов, вычислить количество нечетных отрицательных элементов и сумму элементов списка, расположенных до последнего элемента, равного нулю

```
# Description: Works with an integer list to:

# 1) Count negative odd elements.
# 2) Find the sum of elements before the last zero.
```

```python
from utils import get_int
from init_module import choose_init_method

def display_list(lst: list[int]) -> None:
"""
Print all elements of *lst* in a readable format.

Args:
lst: The list to display.
"""
if not lst:
print(" (the list is empty)")
return
print(" Elements:", ", ".join(str(e) for e in lst))

def validate_list(lst: list[int]) -> bool:
"""
Verify that *lst* is non-empty and contains only integers.

Args:
lst: The list to validate.

Returns:
True if valid, False otherwise.

Raises:
TypeError: If *lst* is not a list.
"""
if not isinstance(lst, list):
raise TypeError("Expected a list.")
if not lst:
return False
return all(isinstance(x, int) for x in lst)

def count_negative_odd(lst: list[int]) -> int:
"""
Count elements that are both negative and odd.

Args:
lst: List of integers.
```

```
Returns:
Number of negative odd elements.
"""
return sum(1 for x in lst if x < 0 and x % 2 != 0)

def sum_before_last_zero(lst: list[int]) -> int | None:
"""
Return the sum of all elements that appear before the last zero.

Args:
lst: List of integers.

Returns:
Sum of elements before the last zero element, or None if no zero exists.
Returns 0 if the last zero is the first element.
"""
last_zero_idx = None
for i in range(len(lst) - 1, -1, -1):
if lst[i] == 0:
last_zero_idx = i
break

if last_zero_idx is None:
return None

return sum(lst[:last_zero_idx])

def task5() -> None:
"""
Business function for Task 5.

Lets the user choose the list size and initialization method,
then computes and prints:
1) The count of negative odd elements.
2) The sum of elements before the last zero.
"""
print(" TASK 5 — Integer list processing")

n = get_int(" Enter the number of list elements: ", min_val=1, max_val=10_000)

my_list: list[int] = []
choose_init_method(my_list, n)
```

```python
print("\n Entered list:")
display_list(my_list)

try:
if not validate_list(my_list):
print(" Error: the list is empty or contains invalid data.")
return
except TypeError as exc:
print(f" Validation error: {exc}")
return

# --- Result 1 ---
neg_odd = count_negative_odd(my_list)
print(f"\n 1) Negative odd elements: {neg_odd}")

# --- Result 2 ---
s = sum_before_last_zero(my_list)
if s is None:
print(" 2) The list contains no zeros — sum cannot be computed.")
else:
print(f" 2) Sum of elements before the last zero: {s}")
```

```
Select task (0 to exit): 5
  TASK 5 – Integer list processing
  Enter the number of list elements: 5

Choose initialization method:

1 – Random generator
2 – Manual user input

Your choice (1 or 2): 2
  Enter 5 integers:
    Element [1]: 1
    Element [2]: l
  Error: please enter a valid integer.
    Element [2]: 15
    Element [3]: 1.5
  Error: please enter a valid integer.
    Element [3]: 16
    Element [4]: 18
    Element [5]: -1
  List successfully filled from user input.

  Entered list:
  Elements: 1, 15, 16, 18, -1

  1) Negative odd elements: 1
  2) The list contains no zeros – sum cannot be computed.
```